



NetCentric

Lab2

Done by:

Reem Mohamed Ali 8388

Hour Mohamed 8119

Lab2

Parallel K-Means using Hadoop

1- The unparallelled K-Means pseudo-code:

1. Start timer:

start_time = current time

2.If initial centroids are not given:

Randomly select K points from X as initial centroids

3. Repeat for up to max_iter:

a. Assignment Step:

For each data point x_i :

Assign it to the closest centroid c_j

b. Update Step:

For each cluster j :

Update centroid c_j as the mean of points assigned to it

c. Convergence Check:

If centroid change $<$ epsilon for all centroids:

Break

4. End timer:

end_time = current time

5. Runtime = end_time - start_time

Print or store the runtime

2-MapReduce K-Means algorithm:

- **Overall Algorithm Flow:**
 - **Initialize centroids** by placing centroids.txt in HDFS root (/centroids.txt).
 - For each iteration:
 - Run a MapReduce job.
 - Mapper assigns each point to the nearest centroid.
 - Reducer computes the new centroid by calculating mean for each cluster.
 - New centroids are compared with old ones to check convergence.
 - Iteration stops when either:
 - Maximum number of iterations is reached, or
 - Centroids change less than a small threshold (convergence).
-

1. KMeansMapper.java

Purpose:

Assigns each data point to its nearest centroid.

Main Logic:

- Loads centroids from /centroids.txt during setup().
- In map(), for each line in iris.data:
 - Parses the point.
 - Computes Euclidean distance between the point and each centroid.
 - Emits a key-value pair:
Key = nearest centroid index,
Value = the data point.

Output of Mapper:

(centroid_id, data_point)

2. KMeansReducer.java

Purpose:

Calculates the **new centroid** for each cluster.

Main Logic:

- In `reduce()`, receives all data points assigned to a specific centroid ID.
- Parses and sums all points.
- Calculates the **mean (new centroid)** of those points.
- Emits updated centroid as:
 Key = centroid_id,
 Value = new_centroid_vector

Output of Reducer:

(centroid_id, new_centroid)

3. KMeansDriver.java

Purpose:

Controls the **entire K-Means process**, including:

- Running iterations.
- Reading/writing centroids.
- Checking for **convergence**.

Main Logic:

- Reads initial centroids from `/centroids.txt`.
- Runs the Mapper and Reducer using Job.
- After each iteration:
 - Reads output centroids and compares with old centroids
 - If $\text{change} < \text{threshold}$ → stop (converged).
 - Else → write new centroids back to `/centroids.txt` and repeat.
- Writes final result to `/final_centroids.txt`.

3-The challenges we faced to implement it and how we solved it:

1. Initial Centroid File Not Found in HDFS

- **Problem:**
The program gave an error:
Initial centroids.txt not found in HDFS.
- **Cause:**
The centroids.txt file was either not uploaded to HDFS or was uploaded to a different path.
- **Solution:**
Used the command: **hdfs dfs -put -f centroids.txt /**

This ensured the file is in the correct location as expected by the program.

2. Input Path Does Not Exist

- **Problem:**
Input path does not exist: hdfs://localhost:9000/inputKMeans/iris.data
 - **Cause:**
The input dataset was not uploaded to HDFS.
 - **Solution:**
Uploaded the input file using: **hdfs dfs -mkdir /inputKMeans**
Hdfs dfs -put iris.data /inputKMeans/
-

3. Missing Class Error (ClassNotFoundException)

- **Problem:**
Error message:
Exception in thread "main" java.lang.NoClassDefFoundError: KMeansMapper
- **Cause:**
The compiled JAR file did not include all required classes (Mapper, Reducer, Driver)
- **Solution:**
Rebuilt the JAR properly in Eclipse ensuring all .java files were included in the export.

4. JAR File Not Updating with Code Changes

- **Problem:**
Code changes made in Eclipse didn't reflect when running JAR.
 - **Cause:**
JAR wasn't rebuilt after changes.
 - **Solution:**
Always rebuilt JAR (Export → Runnable JAR) after code updates.
-

5. Output Path Already Exists Error

- **Problem:**
Error: org.apache.hadoop.mapred.FileAlreadyExistsException: Output directory already exists.
- **Cause:**
Reusing an output path that was already created by a previous run.
- **Solution:**
Used a **new output folder name** for each run or deleted the old one:

```
hdfs dfs -rm -r /outputKMeans0
```

4-The evaluation results:

Runtime Comparison:

- UnParallel KMeans takes 0.015 sec
- MapReduceKMeans takes 12.5 sec

MapReduce is expected to be slower for small datasets like Iris due to Hadoop overhead. But it's scalable for large datasets.

Clustering Accuracy:

The resulting centroids are the same